

PyTorch

OpenEnv Hackathon



UC Berkeley
Center for Responsible,
Decentralized Intelligence



OpenPipe



MERCOR

 fleet AI



Snorkel

scale

Patronus AI



Hallucinate

scaler ai labs



Build environments in minutes. Train agents with any framework.

Deploy anywhere. Same API everywhere.



GitHub

github.com/meta-pytorch/OpenEnv



Documentation

huggingface.co/docs/trl/openenv



HuggingFace Spaces

huggingface.co/openenv

`$ pip install openenv-core`

`$ openenv init my_env`

`$ openenv serve my_env`

`$ openenv push`

Main references

OpenEnv Core:

<https://github.com/meta-pytorch/OpenEnv>

OpenEnv-PyTorch Doc:

<https://meta-pytorch.org/OpenEnv/>

OpenEnv HF:

<https://huggingface.co/openenv>

OpenEnv HF Spaces:

<https://huggingface.co/openenv/spaces>

Tutorials:

<https://github.com/meta-pytorch/OpenEnv/tree/main/tutorial>

Training Examples:

<https://github.com/meta-pytorch/OpenEnv/tree/main/tutorial/examples>

Envs examples:

<https://github.com/meta-pytorch/OpenEnv/tree/main/envs>

How to access Infra:

<https://northflank.notion.site/Deploy-AI-projects-with-Northflank-1a76d14c7851805f8a0ecc780fa33547>

TOC:

- **OpenEnv fundamentals, architecture**
- **Local dev, Docker, and HF Spaces deployment**
- **OpenEnv in Practice**
- **Training (TRL & Unsloth)**
- **How-to-Access-Infrastructure**

- **OpenEnv fundamentals, architecture**
- Local dev, Docker, and HF Spaces deployment
- OpenEnv in Practice
- Training (TRL & Unsloth)
- How-to-Access-Infrastructure

INTRODUCTION

What is OpenEnv?

An **end-to-end framework** for creating, deploying, and training with **isolated execution environments** for agentic reinforcement learning. Built on **Gymnasium-style APIs** that work identically across training, evaluation, and production.



Create environments with simple Python classes



Deploy anywhere — Docker, HuggingFace, Cloud



Train with TRL, Unsloth, torchforge, and more



Isolate agents in sandboxed containers

ARCHITECTURE OVERVIEW

Gym-like WebSocket`reset() · step() · state()`

Training Orchestration

MCP Protocol`tools/list · tools/call`

Agent Interaction

↓ Docker Isolation ↓

echo_env

coding_env

openspiel_env

browsergym_env

git_env

atari_env

snake_env

carla_env

THE PROBLEM

Why Do We Need OpenEnv?

🔧 Fragmented Ecosystem 01

PROBLEM

Every RL project builds its own environment wrapper. Custom adapters for each framework. No reuse.

OPENENV SOLUTION

One standard interface. Build once, use everywhere — TRL, Unsloth, torchforge, SkyRL.

⚡ Train vs Prod Gap 02

PROBLEM

Training environments differ from production. Agents break when deployed. Lifecycle mismatch.

OPENENV SOLUTION

Identical interface for training, eval, and production. Minimize lifecycle deltas.

🔒 No Isolation 03

PROBLEM

Agents run code in the host. One bad action can corrupt the entire system. No sandboxing.

OPENENV SOLUTION

Every environment runs in an isolated Docker container. Agents can't escape their sandbox.

🚀 Scaling Pain 04

PROBLEM

Running 64+ environments in parallel requires custom infrastructure every time.

OPENENV SOLUTION

Built-in scaling: Docker Swarm, cloud sandboxes. EnvPool for batch collection.

Core philosophy: Tools that work for humans should work for agents — and vice versa.

BACKGROUND

Reinforcement Learning Basics



$$G = \sum \gamma^t \cdot r_t \quad (\text{maximize cumulative reward})$$

- **Agent**

The learner. Takes `observations` and outputs `actions`. In LLM RL, this is the language model itself.

- **Environment**

The world. Receives an action, updates state, returns `observation`, `reward`, and `done` flag.

- **Action**

What the agent does. In agentic RL: tool calls, code execution, text responses. Sent via `env.step(action)`.

- **Observation**

What the agent sees after acting. Contains `reward` (scalar feedback) and `done` (episode over?).

- **Episode**

One full interaction: `reset()` → multiple `step()` calls → `done=True`. Then start a new episode.

CORE CONCEPT

The RL Training Loop in OpenEnv

train.py

```

1  import asyncio
2  from hack_env import HackEnv
3
4  async def train_episode(env, model):
5      # 1. Start a new episode
6      obs = await env.reset()
7
8      trajectory = []
9      while not obs.done:
10         # 2. Agent decides action
11         action = model.predict(obs)
12
13         # 3. Take action in environment
14         result = await env.step(action)
15
16         # 4. Collect trajectory
17         trajectory.append((
18             obs, action,
19             result.reward,
20             result.observation
21         ))
22         obs = result.observation
23
24     # 5. Compute loss & update model
25     loss = grpo_loss(trajectory)
26     loss.backward()

```

1 Reset Environment

Start fresh episode. Environment returns initial observation.

```
obs = await env.reset()
```

2 Agent Decides

Model generates action from observation (LLM generates text/tool calls).

```
action = model.predict(obs)
```

3 Step Environment

Environment processes action, returns observation + reward + done.

```
result = await env.step(action)
```

4 Collect Trajectory

Store (state, action, reward) tuples for policy gradient computation.

5 Compute Loss

Use GRPO, PPO, or REINFORCE to compute policy gradient loss.

```
loss = grpo_loss(trajectory)
```

6 Update Model

Backpropagate and update weights. Repeat for next episode.

```
loss.backward(); optimizer.step()
```


The Dual API Design

Two interfaces for two boundaries — training orchestration never leaks into agent interaction

🔒 Key Invariant

Agents can NEVER call reset(). If agents could undo consequences, training breaks.

🧠 Training Loop

RL framework (TRL, torchforge)
`env.reset()` • `env.step()` • `env.state()`

🧠 Agent (LLM)

The model being trained
`tools/List` • `tools/call`

↓ WebSocket ↓

Gym-like API (WebSocket)

Simulation control: `reset`, `step`, `state`, `close`
`POST /reset` • `POST /step` • `WS /ws`

MCP Protocol (JSON-RPC)

Agent tools: domain-specific actions
`POST /mcp` • `WS /mcp`

↓ FastAPI + HTTPEnvServer ↓

Environment Instance

Your custom environment logic
`class MyEnv(MCPEnvironment)`

Rubric System

Reward computation (nn.Module-style)
`Sequential`, `WeightedSum`, `Gate`

↓ Container Boundary ↓

🐳 Docker Container (Isolated Sandbox)

Each environment runs in complete isolation • Local Docker • Swarm • Kubernetes • Dayfona Cloud

SIMULATION mode — full API for training

PRODUCTION mode — safe API only (no reset/step)

⌚ Two Time Models

Simulation:

time freezes between steps

Production:

real-time, no resets

Creating a Server Environment

Three files is all you need: models, environment logic, and an app entry point

ENVIRONMENT STRUCTURE

hack_env/	
__init__.py	exports
models.py	types
client.py	client
openenv.yaml	manifest
server/	
hack_environment.py	logic
app.py	entry
requirements.txt	
Dockerfile	

Quick scaffold:

```
$ openenv init hack_env
```

models.py

Data Types

```
from openenv import Action, Observation, State

class HackAction(Action):
    command: str          # e.g. "nmap -sV target"
    target: str = ""

class HackObservation(Observation):
    output: str = ""      # command output
    flags_found: list[str] = []

class HackState(State):
    vulnerabilities_found: int = 0
    total_flags: int = 5
```

server/hack_environment.py

Core Logic

```
from openenv import MCPEnvironment
from mcp.server.fastmcp import FastMCP

class HackEnvironment(MCPEnvironment):
    def __init__(self):
        mcp = FastMCP("hack_env")
        @mcp.tool
        def scan_port(target: str, port: int) → str: ...
        @mcp.tool
        def exploit_vuln(vuln_id: str) → dict: ...
        super().__init__(mcp)

    def reset(self, **kw) → HackObservation: ...
    def step(self, action, **kw) → HackObservation: ...
```

server/app.py

Entry Point

```
from openenv import create_app
from .hack_environment import HackEnvironment
from ..models import HackAction, HackObservation

app = create_app(HackEnvironment, HackAction, HackObservation, env_name="hack_env")
# Pass class (factory), NOT instance - each WS session gets its own env!
```

HackEnv — Full Server Implementation

hack_env

server/hack_environment.py

```

from uuid import uuid4
from openenv import MCPEnvironment
from mcp.server.fastmcp import FastMCP
from ..models import HackObservation, HackState

class HackEnvironment(MCPEnvironment):
    # Hidden CTF flags the agent must discover
    FLAGS = {"web_vuln", "sql_inject", "priv_esc",
            "exposed_api", "weak_creds"}

    def __init__(self):
        mcp = FastMCP("hack_env")

        @mcp.tool
        def scan_port(host: str, port: int) → str:
            """Scan a port on the target host."""
            if port == 8080:
                self._found.add("exposed_api")
                return "OPEN - HTTP API (no auth)"
            return f"Port {port}: closed"

        @mcp.tool
        def check_sql(url: str, payload: str) → str:
            """Test SQL injection on endpoint."""
            if " OR" in payload:
                self._found.add("sql_inject")
                return "VULNERABLE: SQL injection!"
            return "No vulnerability found"

        @mcp.tool
        def brute_login(user: str, pwd: str) → str:
            """Attempt login with credentials."""
            if pwd in ["admin", "password", "123456"]:
                self._found.add("weak_creds")
                return "LOGIN SUCCESS (weak password!)"
            return "Login failed"

    super().__init__(mcp)
    self._state = HackState()
    self._found = set()

```

server/hack_environment.py (continued)

```

def reset(self, seed=None, episode_id=None,
          **kwargs) → HackObservation:
    self._found = set()
    self._state = HackState(
        episode_id=episode_id or str(uuid4()),
        step_count=0,
        total_flags=len(self.FLAGS)
    )
    return HackObservation(
        output="Target system ready. Find 5 flags.",
        done=False, reward=0.0
    )

def step(self, action, timeout_s=None,
          **kwargs) → HackObservation:
    self._state.step_count += 1
    obs = super().step(action, timeout_s)

    # Compute reward: +1 per new flag
    new_flags = len(self._found)
    reward = new_flags / len(self.FLAGS)
    done = new_flags == len(self.FLAGS)

    self._state.vulnerabilities_found = new_flags
    return HackObservation(
        output=obs.output,
        flags_found=list(self._found),
        reward=reward, done=done
    )

@property
def state(self) → HackState:
    return self._state

```

Key patterns: MCP tools defined in `__init__` with `@mcp.tool`. Environment computes rewards internally (never the agent). State tracks episode progress.

Creating a Client Environment

Type-safe client that connects to any OpenEnv server via WebSocket

client.py

```
from openenv import EnvClient, StepResult
from .models import (
    HackAction, HackObservation, HackState
)

class HackEnv(
    EnvClient[HackAction, HackObservation, HackState]
):
    """Client for the HackEnv CTF environment."""

    def _step_payload(self, action: HackAction):
        # Serialize action to JSON for WebSocket
        return action.model_dump()

    def _parse_result(self, payload):
        # Deserialize server response
        obs = HackObservation(**payload)
        return StepResult(
            observation=obs,
            reward=obs.reward,
            done=obs.done
        )

    def _parse_state(self, payload):
        return HackState(**payload)

# --- Usage: Async (recommended) ---
async with HackEnv(
    base_url="http://localhost:8000"
) as env:
    obs = await env.reset()
    result = await env.step(action)

# --- Usage: Sync wrapper ---
with HackEnv(base_url="...").sync() as env:
    obs = env.reset()
    result = env.step(action)
```

Generic Type Safety

`EnvClient[ActT, ObsT, StateT]` ensures compile-time type checking. All validated via Pydantic.

Three Abstract Methods

Implement `_step_payload` (serialize), `_parse_result` (deserialize obs), and `_parse_state` (deserialize state).

Connection Modes

async with env:

`env.sync()`

`from_docker_image()`

`from_env("org/env")`

MCP Tool Client (Alternative)

For quick prototyping, use `MCPToolClient` directly — no subclass needed. Discover and call tools dynamically.

Factory Methods

`HackEnv.from_docker_image("hack-env:latest")`
`HackEnv.from_env("openenv/hack-env")`
 Auto-starts containers, waits for health check, connects.

LIVE EXAMPLE

Working with Server & Client

run_hackenv.py

```
import asyncio
from hack_env import HackEnv
from openenv import MCPToolClient

async def main():
    # Option 1: Typed client
    async with HackEnv(
        base_url="http://localhost:8080"
    ) as env:

        # Reset - start new episode
        obs = await env.reset()
        print(obs.output)

        # Step 1: Scan for open ports
        result = await env.step(
            CallToolAction(
                name="scan_port",
                arguments={
                    "host": "target.local",
                    "port": 8080
                }
            )
        )
        print(result.observation.output)
        print(f"Reward: {result.reward}")

        # Step 2: Test SQL injection
        result = await env.step(
            CallToolAction(
                name="check_sql",
                arguments={
                    "url": "/api/users",
                    "payload": "' OR 1=1--"
                }
            )
        )

        # Check state anytime
        state = await env.state()
        print(f"Found {state.vulnerabilities_found}/{state.total_flags}")

asyncio.run(main())
```

Terminal - Server

```
$ uvicorn server.app:app --port 8080
INFO: Uvicorn running on http://0.0.0.0:8080
INFO: WS connection opened (session: a1b2c3)
INFO: Environment created for session a1b2c3
INFO: reset() called - new episode started
INFO: step() - tool: scan_port(8080)
INFO: FLAG FOUND: exposed_api
INFO: step() - tool: check_sql("' OR 1=1--")
INFO: FLAG FOUND: sql_inject
```

↓

Terminal - Client Output

```
$ python run_hackenv.py
Target system ready. Find 5 flags.

OPEN - HTTP API (no auth)
Reward: 0.2

VULNERABLE: SQL injection!
Reward: 0.4
Flags: ['exposed_api', 'sql_inject']

Found 2/5 vulnerabilities
(episode continues...)
```

Or use MCPToolClient (no subclass needed):

```
async with MCPToolClient(base_url="...") as env:
    tools = await env.list_tools()
    result = await env.call_tool("scan_port",
        host="target.local", port=8080)
```

Deploying Environments



Local Docker

Build and run locally. Each container is isolated with its own filesystem and network.

```
# Build image
$ docker build -t hack-env:latest -f server/Dockerfile .

# Run container
$ docker run -p 8000:8000 hack-env

# Or use openenv CLI
$ openenv build
$ openenv serve hack_env
```

DOCKER



HuggingFace Spaces

Push to HuggingFace for free hosting. Gets a public URL with auto-scaling.

```
# Push to HF Spaces
$ openenv push

# Connect from client
env = HackEnv(base_url=
  "https://openenv-hack-env.hf.space"
)

# Or use from_env factory
env = await HackEnv.from_env(
  "openenv/hack-env")
```

HUGGINGFACE



Cloud Sandboxes

Daytona or Docker Swarm for production-grade isolation and scaling.

```
# Daytona cloud sandbox
env = await HackEnv(
  provider=DaytonaProvider()
)

# Docker Swarm (replicated)
provider = DockerSwarmProvider()
url = provider.start_container(
  "hack-env:latest",
  replicas=4
)
```

DAYTONA

SWARM

● ● ● Dockerfile — Standard Pattern

```
# Stage 1: Build
ARG BASE_IMAGE=ghcr.io/meta-pytorch/openenv-base:latest
FROM ${BASE_IMAGE} AS builder
WORKDIR /app
COPY . /app/env
RUN uv sync --frozen --no-editable
```

```
# Stage 2: Runtime
FROM ${BASE_IMAGE}
COPY --from=builder /app/env /app/env
HEALTHCHECK CMD python -c "...
CMD ["uvicorn", "server.app:app",
  "--host", "0.0.0.0", "--port", "8000"]
```

Multi-Env / Multi-Session / Multi-Worker

One environment = one trajectory. To scale, stack environment instances.

① Concurrent Sessions

Multiple WebSocket connections on one server

SINGLE SERVER, N SESSIONS

FastAPI Server (:8000)

S1

S2

S3

S4

```
# Server-side config
app = create_app(
    HackEnv,
    max_concurrent_envs=4,
    session_timeout=300
)

# Requires:
SUPPORTS_CONCURRENT_SESSIONS = True
```

② Docker Swarm Replicas

Horizontal scaling with load balancing

LOAD BALANCER → N REPLICAS

Docker Swarm Mesh

R1

R2

R3

R4

```
provider = DockerSwarmProvider()
url = provider.start_container(
    "hack-env:latest",
    replicas=4,
    cpu_limit="2.0",
    memory_limit="4g"
)
```

③ Cloud Batch (N Workers)

Parallel isolated sandboxes for batch RL

ASYNCTIO.GATHER → N SANDBOXES

Daytona Cloud

W1

W2

W3

W4

W5

W6

W7

W8

```
# Batch training pattern
results = await asyncio.gather(
    *(run_episode(i)
      for i in range(64))
)

# 64 parallel episodes!
```



Key principle: Environments do not support multiplexed trajectories. To generate batches, stack multiple environment instances. Helpers like **EnvPool** orchestrate batch collection across the stack.

CONNECTING

Loading Environments

Six ways to connect — from local dev to cloud-scale production



Local Server (base_url)

Simplest

Connect to a running server. Start it yourself with `uvicorn` or `openenv serve`.

```
# Async
async with HackEnv(base_url="http://localhost:8000") as env:
    obs = await env.reset()

# Sync wrapper
with HackEnv(base_url="http://localhost:8000").sync() as env:
    obs = env.reset()
```



Docker Image (from_docker_image)

Auto-start

Starts a container from a local Docker image. Auto-finds port, waits for health check, cleans up on close.

```
env = await HackEnv.from_docker_image("hack-env:latest")
async with env:
    obs = await env.reset()
# Container auto-removed on exit
```



HuggingFace Hub (from_env)

Auto-pull

Pulls Docker image from HuggingFace registry, starts container, and connects. One-liner for any published env.

```
# Docker mode (default) - pulls from registry.hf.space
env = await HackEnv.from_env("openenv/hack-env")

# UV mode - clones repo and runs locally
env = await HackEnv.from_env("openenv/hack-env", use_docker=False)
```



UV Provider (Local Dev)

No Docker

Runs environment directly via `uv run`. No container overhead. Hot reload for development.

```
from openenv.containers.runtime import UVProvider

provider = UVProvider(project_path="./hack_env", reload=True)
url = provider.start(port=8000)
env = HackEnv(base_url=url, provider=provider)
```



Docker Swarm (Replicated)

Scaling

Creates replicated services with load balancing. Ideal for parallel batch RL training.

```
from openenv.containers.runtime import DockerSwarmProvider

provider = DockerSwarmProvider()
url = provider.start_container(
    "hack-env:latest", replicas=4,
    cpu_limit="2.0", memory_limit="4g"
)
```



Daytona Cloud (Sandboxes)

Cloud

Fully isolated cloud sandboxes. Each environment in its own VM. Signed preview URLs.

```
from openenv.containers.runtime import DaytonaProvider

provider = DaytonaProvider(api_key="...")
url = provider.start_container("hack-env:latest")
env = HackEnv(base_url=url, provider=provider)
```



All methods use the same client API. Once connected, `reset()`, `step()`, `state()`, `close()` work identically regardless of how you loaded the environment.

Loading Environments

Four ways to connect — from local development to HuggingFace Hub



Local Server (base_url)

Simplest

Connect to an already running server. Start it with `uvicorn` or `openenv serve`.

```
# Async (recommended)
async with HackEnv(base_url="http://localhost:8000") as env:
    obs = await env.reset()
    result = await env.step(action)

# Sync wrapper
with HackEnv(base_url="http://localhost:8000").sync() as env:
    obs = env.reset()
```



Docker Image (from_docker_image)

Auto-start

Starts a container from a local Docker image. Auto-finds an available port, waits for health check, cleans up on close.

```
env = await HackEnv.from_docker_image("hack-env:latest")

async with env:
    obs = await env.reset()
    result = await env.step(action)
# Container auto-removed on exit
```



HuggingFace Hub (from_env)

Auto-pull

Pulls a Docker image from the HuggingFace registry, starts the container, and connects. One-liner for any published environment.

```
# Docker mode (default) - pulls from registry.hf.space
env = await HackEnv.from_env("openenv/hack-env")

# UV mode - clones the repo and runs locally
env = await HackEnv.from_env(
    "openenv/hack-env", use_docker=False
)
```



UV Provider (Local Dev)

No Docker

Runs the environment directly via `uv run`. No container overhead. Supports hot reload for development.

```
from openenv.containers.runtime import UVProvider

provider = UVProvider(
    project_path="./hack_env", reload=True
)
url = provider.start(port=8000)
env = HackEnv(base_url=url, provider=provider)
```



All methods use the same client API. Once connected, `reset()`, `step()`, `state()`, `close()` work identically regardless of how you loaded the environment.

- OpenEnv fundamentals, architecture
- **Local dev, Docker, and HF Spaces deployment**
- OpenEnv in Practice
- Training (TRL & Unsloth)
- How-to-Access-Infrastructure

Every HF Space provides three components:

HF Spaces are the infrastructure for OpenEnv environments

Every HF Space provides three things that OpenEnv environments need:

Component	What it provides	How to access	Used as
Server	Running environment endpoint	<code>https://<username>-<space-name>.hf.space</code>	Agent and Public API
Repository	Installable Python package	<code>pip install git+https://huggingface.co/spaces/<username>-<space-name></code>	Code and client
Registry	Docker container image	<code>docker pull registry.hf.space/<username>-<space-name>:latest</code>	Deployment

This means a single Space deployment gives you all the components you need to use an environment in training.

1. Server: A running environment endpoint

```
from echo_env import EchoEnv, EchoAction

# Connect directly to the running Space (WebSocket under the hood)
# Async (recommended):
async with EchoEnv(base_url="https://openenv-echo-env.hf.space") as client:
    result = await client.reset()
    result = await client.step(EchoAction(message="Hello"))

# Sync (using .sync() wrapper):
with EchoEnv(base_url="https://openenv-echo-env.hf.space").sync() as client:
    result = client.reset()
    result = client.step(EchoAction(message="Hello"))
```

Example: Checking if the space is running
`curl https://openenv-echo-env.hf.space/health`
`# {"status": "healthy"}`

Endpoints available:

Endpoint	Protocol	Description
<code>/ws</code>	WebSocket	Persistent session (used by client)
<code>/health</code>	HTTP GET	Health check
<code>/reset</code>	HTTP POST	Reset environment (stateless)
<code>/step</code>	HTTP POST	Execute action (stateless)
<code>/state</code>	HTTP GET	Get current state
<code>/docs</code>	HTTP GET	OpenAPI documentation
<code>/web</code>	HTTP GET	Interactive web UI

2. Repository: Installable Python package

Every Space is a Git repository. OpenEnv environments include a `pyproject.toml`, making them pip-installable directly from the Space URL.

```
# Install client package from Space
pip install git+https://huggingface.co/spaces/openenv/echo-env
```

This installs:

- **Client class** (`EchoEnv`) — Handles HTTP/WebSocket communication
- **Models** (`EchoAction` , `EchoObservation`) — Typed action and observation classes
- **Utilities** — Any helper functions the environment provides

After installation:

```
from envs.echo_env import EchoEnv, EchoAction, EchoObservation

# Now you have typed classes for the environment
action = EchoAction(message="Hello")
```

3. Registry: Docker container image

```
# Pull the image
docker pull registry.hf.space/openenv-echo-env:latest

# Run locally on port 8001
docker run -d -p 8001:8000 registry.hf.space/openenv-echo-env:latest
```

[illegible]

A screenshot of the Docker Desktop 'Run locally' dialog box. The window has a title bar with a hamburger menu icon, the text 'Run locally', and a close button. The main content area is divided into sections. The first section, 'Run this Space with Docker', contains a terminal window with the command: `docker run -it -p 7860:7860 --platform=linux/amd64 \ registry.hf.space/crashbandicoot2-doom-env:latest`. To the right of the command is a copy icon. The second section, 'Available versions', features a dropdown menu currently showing 'latest'. Below this, a message states: 'You need to switch on the new containerd engine in Docker for pulling and storing images. This will be the new Docker default in the future. [Read more](#)'. The final section, 'Quick Links', lists two items: 'How to run any Space as a Docker image' and 'Docs on the Docker Hub (docker.com)', each preceded by a circular icon.

Typical workflow:

```
import asyncio
from echo_env import EchoEnv, EchoAction

async def main():
    # Development: connect to remote Space
    async with EchoEnv(base_url="https://openenv-echo-env.hf.space") as client:
        result = await client.reset()

    # Production: run locally for speed
    # docker run -d -p 8001:8000 registry.hf.space/openenv-echo-env:latest
    async with EchoEnv(base_url="http://localhost:8001") as client:
        result = await client.reset()

    # Or let the client manage Docker for you
    client = await EchoEnv.from_env("openenv/echo-env") # Auto-pulls and runs
    async with client:
        result = await client.reset()

asyncio.run(main())

# For sync usage, use the .sync() wrapper:
with EchoEnv(base_url="http://localhost:8001").sync() as client:
    result = client.reset()
```

Local Development

- Clone and run the environment locally

```
# Clone from HF Space
git clone https://huggingface.co/spaces/burtenshaw/openenv-benchmark
cd openenv-benchmark

# Install in editable mode
uv sync

# Start server
uv run server

# Run isolated from remote Space
uv run --isolated --project https://huggingface.co/spaces/burtenshaw/openenv-benchmark server
```


Local Development

- Uvicorn directly in python

```
# Full control over uvicorn options
uvicorn benchmark.server.app:app --host "$HOST" --port "$PORT" --workers "$WORKERS"

# With reload for development
uvicorn benchmark.server.app:app --host 0.0.0.0 --port 8000 --reload

# Multi-Worker Mode For better concurrency:
uvicorn benchmark.server.app:app --host 0.0.0.0 --port 8000 --workers 4
```



Local Development

- Run the environment locally from the space

```
# Run the environment locally from the space  
docker run -d -p 8000:8000 registry.hf.space/openenv-echo-env:latest
```



Build Image

```
# Clone from HF Space
git clone https://huggingface.co/spaces/burtenshaw/openenv-benchmark
cd openenv-benchmark

# Using OpenEnv CLI (recommended)
openenv build -t openenv-benchmark:latest

# Or with Docker directly
docker build -t openenv-benchmark:latest -f server/Dockerfile .
```

Run Container

```
# Basic run
docker run -d -p 8000:8000 my-env:latest

# With environment variables
docker run -d -p 8000:8000 \
  -e WORKERS=4 \
  -e MAX_CONCURRENT_ENVS=100 \
  my-env:latest

# Named container for easy management
docker run -d --name my-env -p 8000:8000 my-env:latest
```

Deploy with CLI

Initialize a new environment

```
openenv init my_env  
cd my_env
```



Develop and Deploy to HF Spaces

```
cd my_env  
  
# Deploy to your namespace  
openenv push  
  
# Deploy to specific repo  
openenv push --repo-id username/my-env  
  
# Deploy as private  
openenv push --repo-id username/my-env --private
```



- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- **OpenEnv in Practice**
- Training (TRL & Unsloth)
- How-to-Access-Infrastructure

OpenEnv in Practice

```
Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
● > uv venv
Using CPython 3.11.14
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate

Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
●
Downloads/Projects/SF_Hackathon_OpenEnv via 🐍 v3.11.14 (SF_Hackathon_OpenEnv) on ☁ (us-east-1)
● > uv pip list

Downloads/Projects/SF_Hackathon_OpenEnv via 🐍 v3.11.14 (SF_Hackathon_OpenEnv) on ☁ (us-east-1)
● > uv pip install openenv-core
Resolved 86 packages in 309ms
Installed 86 packages in 262ms
```

Use uv venv

Or

```
Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
○ > conda create -n openenv_hackathon python=3.12

Downloads/Projects/SF_Hackathon_OpenEnv on ☁ (us-east-1)
● > conda activate openenv_hackathon
(openenv_hackathon)
Downloads/Projects/SF_Hackathon_OpenEnv via 🍷 openenv_hackathon on ☁ (us-east-1)
○ >

(openenv_hackathon)
Downloads/Projects/SF_Hackathon_OpenEnv via 🍷 openenv_hackathon on ☁ (us-east-1)
● > uv pip install openenv-core
Using Python 3.12.12 environment at: /Users/alisol/miniconda3/envs/openenv_hackathon
Resolved 85 packages in 833ms
Prepared 30 packages in 4.37s
Installed 84 packages in 453ms
```

Use conda env

Initialize an Environment

Downloads/Projects/SF_Hackathon_OpenEnv via ©openenv_hackathon on ☁ (us-east-1)

```
> openenv init HackEnv101_AlirezaShamsoshoara
```

Creating OpenEnv environment 'HackEnv101_AlirezaShamsoshoara'...

✓ Created 11 files

Generating uv.lock...

✓ Generated uv.lock

Environment created successfully at:

/Users/alisol/Downloads/Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara

Next steps:

```
cd /Users/alisol/Downloads/Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara
```

```
# Edit your environment implementation in server/HackEnv101_AlirezaShamsoshoara_environment.py
```

```
# Edit your models in models.py
```

```
# Install dependencies: uv sync
```

```
# To integrate into OpenEnv repo:
```

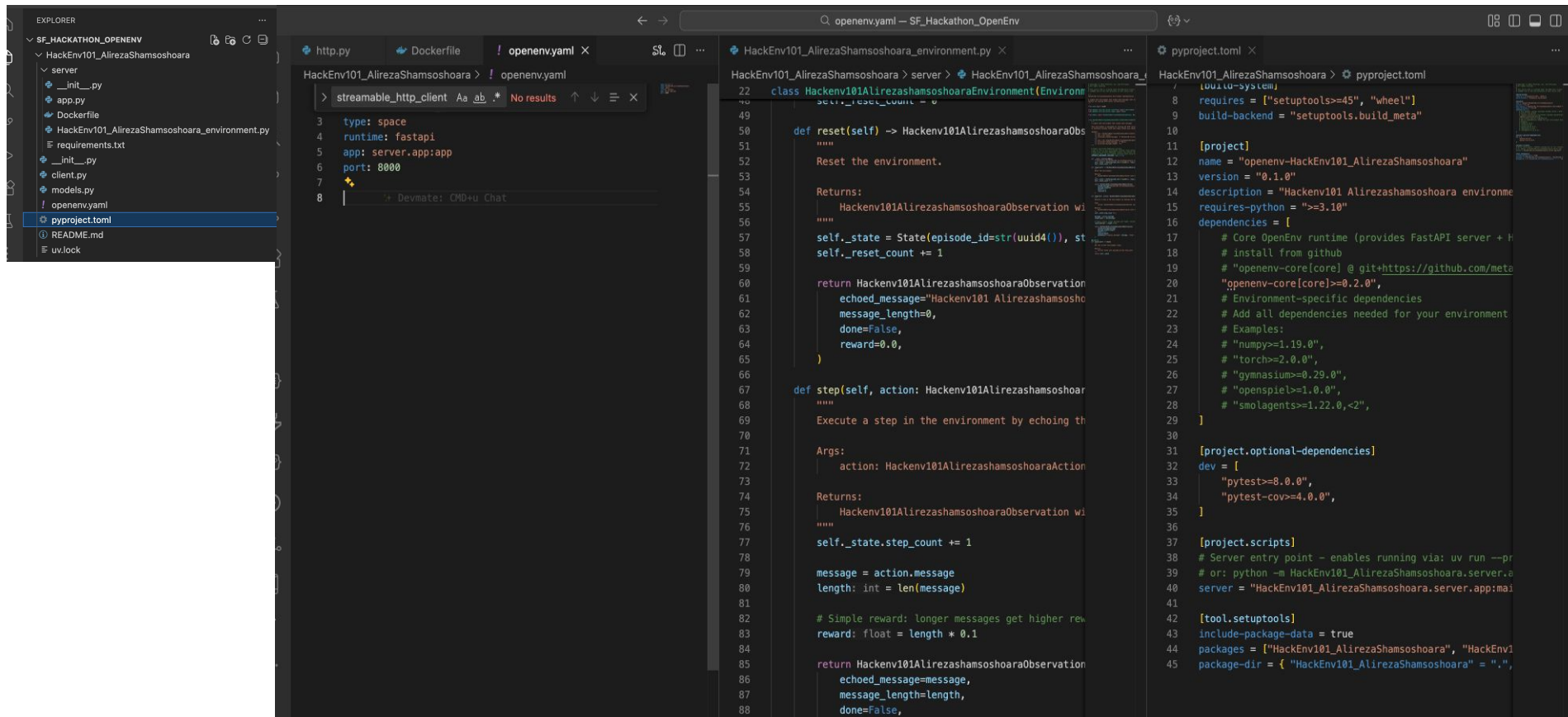
```
# 1. Copy this directory to <repo_root>/envs/HackEnv101_AlirezaShamsoshoara_env
```

```
# 2. Build from repo root: docker build -t HackEnv101_AlirezaShamsoshoara_env:latest -f  
envs/HackEnv101_AlirezaShamsoshoara_env/server/Dockerfile .
```

```
# 3. Run your image: docker run -p 8000:8000 HackEnv101_AlirezaShamsoshoara_env:latest  
(openenv_hackathon)
```

Downloads/Projects/SF_Hackathon_OpenEnv via ©openenv_hackathon on ☁ (us-east-1) took 2s

Initialize an Environment



The image shows a VS Code editor with three tabs open: `openenv.yaml`, `HackEnv101_AlirezaShamsoshoara_environment.py`, and `pyproject.toml`.

EXPLORER (Left Panel):

- SF_HACKATHON_OPENENV
 - HackEnv101_AlirezaShamsoshoara
 - server
 - __init__.py
 - app.py
 - Dockerfile
 - HackEnv101_AlirezaShamsoshoara_environment.py
 - requirements.txt
 - __init__.py
 - client.py
 - models.py
 - openenv.yaml
 - pyproject.toml**
 - README.md
 - uv.lock

openenv.yaml (Middle Panel):

```
streamable_http_client Aa ab * No results ↑ ↓ ≡ ×
```

```
3 type: space
4 runtime: fastapi
5 app: server.app:app
6 port: 8000
7
8 |
```

HackEnv101_AlirezaShamsoshoara_environment.py (Middle Panel):

```
22 class Hackenv101AlirezashamsoshoaraEnvironment(Environ
    self._reset_count = 0

    def reset(self) -> Hackenv101AlirezashamsoshoaraObs
        """
        Reset the environment.

        Returns:
            Hackenv101AlirezashamsoshoaraObservation w1
        """
        self._state = State(episode_id=str(uuid4()), st
        self._reset_count += 1

        return Hackenv101AlirezashamsoshoaraObservation
            echoed_message="Hackenv101 Alirezashamsosho
            message_length=0,
            done=False,
            reward=0.0,
        )

    def step(self, action: Hackenv101Alirezashamsoshoara
        """
        Execute a step in the environment by echoing th

        Args:
            action: Hackenv101AlirezashamsoshoaraAction

        Returns:
            Hackenv101AlirezashamsoshoaraObservation w1
        """
        self._state.step_count += 1

        message = action.message
        length: int = len(message)

        # Simple reward: longer messages get higher rew
        reward: float = length * 0.1

        return Hackenv101AlirezashamsoshoaraObservation
            echoed_message=message,
            message_length=length,
            done=False,
```

pyproject.toml (Right Panel):

```
8 requires = ["setuptools>=45", "wheel"]
9 build-backend = "setuptools.build_meta"

10
11 [project]
12 name = "openenv-HackEnv101_AlirezaShamsoshoara"
13 version = "0.1.0"
14 description = "Hackenv101 Alirezashamsoshoara environme
15 requires-python = ">=3.10"
16 dependencies = [
17     # Core OpenEnv runtime (provides FastAPI server + H
18     # install from github
19     # "openenv-core[core] @ git+https://github.com/meta
20     "openenv-core[core]>=0.2.0",
21     # Environment-specific dependencies
22     # Add all dependencies needed for your environment
23     # Examples:
24     # "numpy>=1.19.0",
25     # "torch>=2.0.0",
26     # "gymnasium>=0.29.0",
27     # "openspiel>=1.0.0",
28     # "smolagents>=1.22.0,<2",
29 ]

30
31 [project.optional-dependencies]
32 dev = [
33     "pytest>=8.0.0",
34     "pytest-cov>=4.0.0",
35 ]

36
37 [project.scripts]
38 # Server entry point - enables running via: uv run --pr
39 # or: python -m HackEnv101_AlirezaShamsoshoara.server.a
40 server = "HackEnv101_AlirezaShamsoshoara.server.app:mai
41
42 [tool.setuptools]
43 include-package-data = true
44 packages = ["HackEnv101_AlirezaShamsoshoara", "HackEnv1
45 package-dir = { "HackEnv101_AlirezaShamsoshoara" = ".",
```


Develop and Push your created environment

```
(openenv_hackathon)
Downloads/Projects/SF_Hackathon_OpenEnv via ☺ openenv_hackathon on ☁ (us-east-1)
• > cd HackEnv101_AlirezaShamsoshoara
(openenv_hackathon)
Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara is 📦 v0.1.0 via 🐙 v3.12.12 via ☺ openenv_hackathon on ☁ (us-east-1)
• > openenv push
Validating OpenEnv environment in /Users/alisol/Downloads/Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara...
  ⚠ Recommended directory missing: outputs/
  ✓ Found OpenEnv environment: HackEnv101_AlirezaShamsoshoara
  ✓ Authenticated as: Crashbandicoote2
Preparing files for Hugging Face deployment (with web interface)...
Moved Dockerfile to repository root for deployment
  ✓ Updated Dockerfile: enabled web interface
Creating/verifying space: Crashbandicoote2/HackEnv101_AlirezaShamsoshoara
  ✓ Space Crashbandicoote2/HackEnv101_AlirezaShamsoshoara is ready
Uploading files to Crashbandicoote2/HackEnv101_AlirezaShamsoshoara...
  ✓ Upload completed successfully
Space URL: https://huggingface.co/spaces/Crashbandicoote2/HackEnv101\_AlirezaShamsoshoara

  ✓ Deployment complete!
Visit your space at: https://huggingface.co/spaces/Crashbandicoote2/HackEnv101\_AlirezaShamsoshoara
(openenv_hackathon)
Projects/SF_Hackathon_OpenEnv/HackEnv101_AlirezaShamsoshoara is 📦 v0.1.0 via 🐙 v3.12.12 via ☺ openenv_hackathon on ☁ (us-east-1) took 6s
○ > █
```

Your Env on HF Spaces

The screenshot shows the Hugging Face Spaces interface for a custom environment. The top navigation bar includes 'Spaces', the space name 'Crashbandicoot2/HackEnv101_AlirezaShamsoshoara', and status indicators like 'like 0' and 'Running'. The main interface is divided into two panels: 'HumanAgent Interface' on the left and 'State Observer' on the right.

HumanAgent Interface:

- Take Action:** A section with a 'Message *:' label, an input field containing 'Enter message...', and a 'Step' button. Below the input field is a hint: 'Message to echo back'.
- Reset Environment:** A button to reset the environment.
- Get State:** A button to get the current state.
- Current State:** A section showing 'Status: Reset', 'Episode ID: ~', and 'Step Count: 0'.

State Observer:

- Current Observation:** A section showing 'No observation yet'.
- Action History:** A section showing 'No actions taken yet'.

Logs: A terminal window at the bottom showing the following output:

```
Build Container </> Logs Endpoint
View Dev Mode Lock scroll Clear ^ x

If the cache and target directories are on different filesystems, hardlinking may not be supported.
If this is intentional, set 'export UV_LINK_MODE=copy' or use '--link-mode=copy' to suppress this warning.
Installed 1 package in 1ms
+ openenv-hackenv101-alirezashamsoshoara==0.1.0 (from file:///app/env)
DONE 1.1s

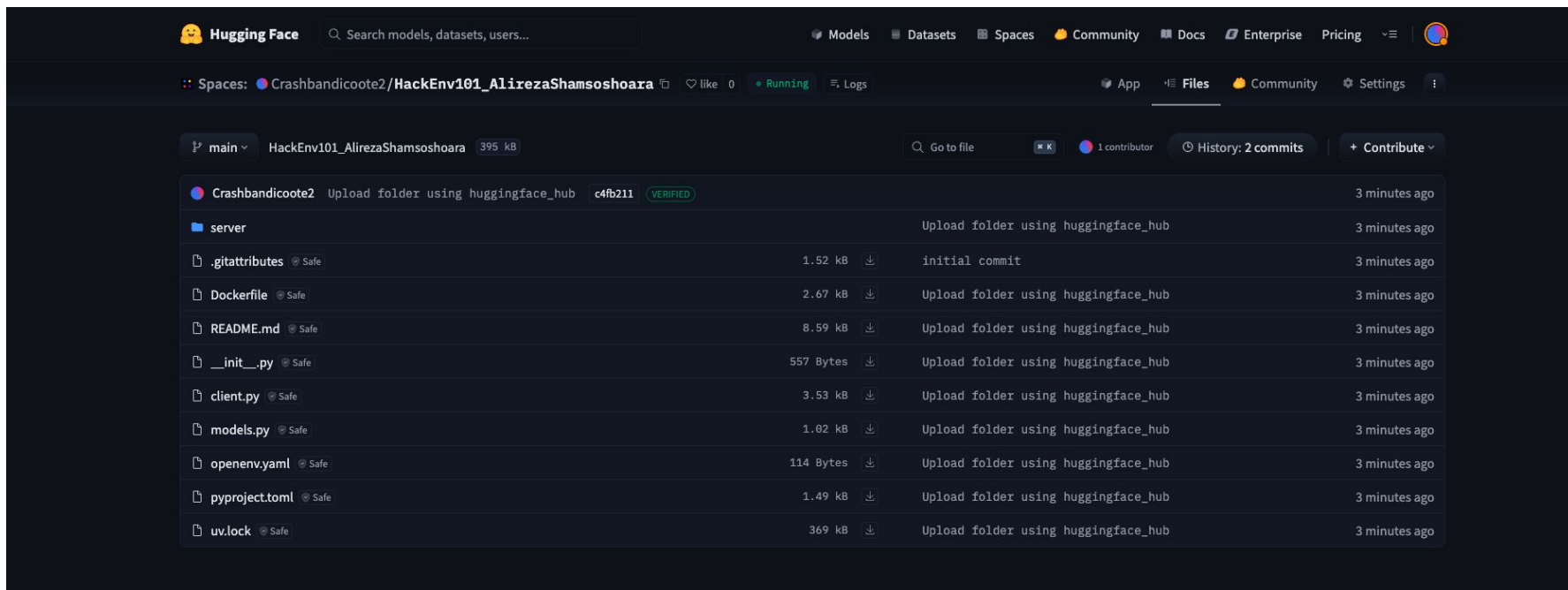
--> COPY --from=builder /app/env/.venv /app/.venv
DONE 0.4s

--> COPY --from=builder /app/env /app/env
DONE 0.5s

--> Pushing image
DONE 3.0s

--> Exporting cache
DONE 3.8s
```

Your Env on HF Spaces



The screenshot displays the Hugging Face interface for a Space named 'HackEnv101_AlirezaShamsoshoara'. The interface is in dark mode and shows the Space is currently 'Running'. The file list includes a 'server' folder and several configuration files like '.gitattributes', 'Dockerfile', 'README.md', and environment files like 'uv.lock'.

Hugging Face Search models, datasets, users... Models Datasets Spaces Community Docs Enterprise Pricing

Spaces: Crashbandicoote2/HackEnv101_AlirezaShamsoshoara like 0 Running Logs App Files Community Settings

main HackEnv101_AlirezaShamsoshoara 395 kB Go to file 1 contributor History: 2 commits + Contribute

File/Folder	Size	Upload Method	Time
Crashbandicoote2		Upload folder using huggingface_hub	3 minutes ago
server		Upload folder using huggingface_hub	3 minutes ago
.gitattributes	1.52 kB	initial commit	3 minutes ago
Dockerfile	2.67 kB	Upload folder using huggingface_hub	3 minutes ago
README.md	8.59 kB	Upload folder using huggingface_hub	3 minutes ago
__init__.py	557 Bytes	Upload folder using huggingface_hub	3 minutes ago
client.py	3.53 kB	Upload folder using huggingface_hub	3 minutes ago
models.py	1.02 kB	Upload folder using huggingface_hub	3 minutes ago
openenv.yaml	114 Bytes	Upload folder using huggingface_hub	3 minutes ago
pyproject.toml	1.49 kB	Upload folder using huggingface_hub	3 minutes ago
uv.lock	369 kB	Upload folder using huggingface_hub	3 minutes ago

Run Locally or Clone the Environment

Run locally

Run this Space with Docker

```
docker run -it -p 7860:7860 --platform=linux/amd64 \  
  registry.hf.space/crashbandicoote2-hackenv101-alirezashamsoshoara:latest
```

Clone this Space repository

HTTPS SSH

```
# Make sure git-xet is installed (https://hf.co/docs/hub/git-xet)  
brew install git-xet  
git xet install
```

```
git clone https://huggingface.co/spaces/Crashbandicoote2/HackEnv101_AlirezaShamsoshoara
```

```
# If you want to clone without large files - just their pointers  
GIT_LFS_SKIP_SMUDGE=1 git clone https://huggingface.co/spaces/Crashbandicoote2/HackEnv101_AlirezaShamsos
```

cURL Homebrew uv uvx

```
# Make sure the hf CLI is installed  
curl -LsSf https://hf.co/cli/install.sh | bash
```

```
# Download the Space  
hf download Crashbandicoote2/HackEnv101_AlirezaShamsoshoara --repo-type=space
```

Available Environments

Growing library of ready-to-use environments — same API across all of them



OpenSpiel

70+ game theory games: chess, poker, blackjack, tic-tac-toe, Go

game-theory



Coding Env

Python code execution with smolagents. Write and run code safely.

code-execution



BrowserGym

Browser automation. Navigate, click, fill forms, scrape pages.

web-browsing



Atari

Classic Atari games via Gymnasium. Breakout, Pong, Space Invaders.

arcade



WebSearch

Web search environment. Query, browse results, extract information.

search



FinRL

Financial RL. Stock trading, portfolio management, market simulation.

finance



CARLA

Autonomous driving simulation with ethical AI benchmarks.

autonomous-driving



Git Env

Git operations environment. Commits, branches, merges, conflict resolution.

devtools



SUMO RL

Traffic signal control simulation. Urban traffic optimization.

traffic



Reasoning Gym

Logic puzzles, math problems, reasoning challenges.

reasoning



Chat Env

Conversational environment for dialogue training and evaluation.

conversation



Snake / Connect4

Classic games for quick prototyping and testing RL pipelines.

classic-games

16+

ENVIRONMENTS

1

UNIFIED API

6+

RL FRAMEWORKS

Same 4 methods everywhere: `reset()` · `step()` · `state()` · `close()`
Build once, train with any framework, deploy anywhere.

- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- OpenEnv in Practice
- **Training (TRL & Unsloth)**
- How-to-Access-Infrastructure

- **Training examples:** <https://github.com/meta-pytorch/OpenEnv/tree/main/tutorial/examples>
- **Unsloth example (2048):** https://github.com/meta-pytorch/OpenEnv/blob/main/tutorial/examples/unsloth_2048.ipynb
- **Wordle example-trl:** <https://github.com/meta-pytorch/OpenEnv/blob/main/tutorial/examples/wordle.py>
- **trl:**
 - **TRL OpenEnv Doc:** <https://huggingface.co/docs/trl/en/openenv>
 - **Sudoku example:** https://github.com/huggingface/trl/blob/main/examples/notebooks/openenv_sudoku_grpo.ipynb
 - **Wordle example:** https://github.com/huggingface/trl/blob/main/examples/notebooks/openenv_wordle_grpo.ipynb
 - **More TRL examples:** <https://github.com/huggingface/trl/tree/main/examples/scripts/openenv>

Training with TRL (GRPO)

HuggingFace TRL integrates natively with OpenEnv environments for GRPO training

trl_grpo_train.py

TRL + OpenEnv

```
from trl import GRPOTrainer, GRPOConfig
from transformers import AutoModelForCausalLM
from hack_env import HackEnv

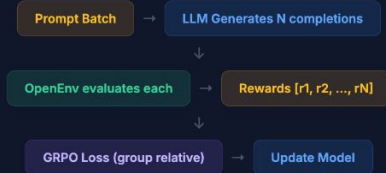
# 1. Load model
model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B-Instruct"
)

# 2. Define reward function using OpenEnv
def openenv_reward(completions, **kwargs):
    rewards = []
    for completion in completions:
        with HackEnv(base_url="...").sync() as env:
            env.reset()
            result = env.step(completion)
            rewards.append(result.reward)
    return rewards

# 3. Configure GRPO training
config = GRPOConfig(
    output_dir="./hack-agent",
    num_generations=4,
    max_completion_length=512,
    per_device_train_batch_size=2,
    learning_rate=5e-6,
)

# 4. Train!
trainer = GRPOTrainer(
    model=model,
    reward_funcs=openenv_reward,
    args=config,
    train_dataset=dataset,
)
trainer.train()
```

GRPO TRAINING FLOW



What is GRPO?

Group Relative Policy Optimization — generates N completions per prompt, ranks them by reward, and updates the policy to prefer higher-reward completions. No critic network needed.

Why OpenEnv + TRL?

OpenEnv provides the `reward_funcs` callback. Each completion is evaluated in an isolated environment. Rewards are computed by the environment's rubric system, not by external heuristics.

Batch Training Pattern

For parallel evaluation, spin up N environment instances. Use `asyncio.gather` or `EnvPool` to evaluate all completions concurrently.

Official Docs

Full TRL + OpenEnv integration guide:
huggingface.co/docs/trl/openenv

Training with Unsloth

Fast RL fine-tuning with Unsloth's optimized training + OpenEnv environments

unsloth_train.py

Unsloth + OpenEnv

```
from unsloth import FastLanguageModel
from trl import GRPOTrainer, GRPOConfig
from hack_env import HackEnv

# 1. Load model with Unsloth (2x faster)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Qwen2.5-7B-Instruct",
    max_seq_length=2848,
    load_in_4bit=True,      # 4-bit quantization
)

# 2. Apply LoRA adapters
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj",
                    "v_proj", "o_proj"],
)

# 3. OpenEnv reward function
def env_reward(completions, **kw):
    rewards = []
    for c in completions:
        with HackEnv("http://localhost:8000"
                     ).sync() as env:
            env.reset()
            r = env.step(c)
            rewards.append(r.reward)
    return rewards

# 4. Train with GRPO
trainer = GRPOTrainer(
    model=model,
    reward_funcs=env_reward,
    args=GRPOConfig(
        learning_rate=5e-6,
        num_generations=4,
```

2x

FASTER TRAINING

70%

LESS MEMORY

4-bit

QUANTIZATION

LoRA

EFFICIENT TUNING

Why Unsloth?

Unsloth provides **2x faster** training and **70% less memory** through custom CUDA kernels. Works as a drop-in replacement — same TRL API, just faster.

The Pattern

1. Load model via `FastLanguageModel` (with 4-bit quantization)
2. Apply LoRA adapters for parameter-efficient training
3. Use OpenEnv as the reward function
4. Train with standard `GRPOTrainer`

Google Colab Ready

Run on a free T4 GPU. Unsloth + OpenEnv Colab notebook available for the 2048 game environment with 20B parameter models.

Also Compatible With

TRL

torchforge

SkyRL

ART

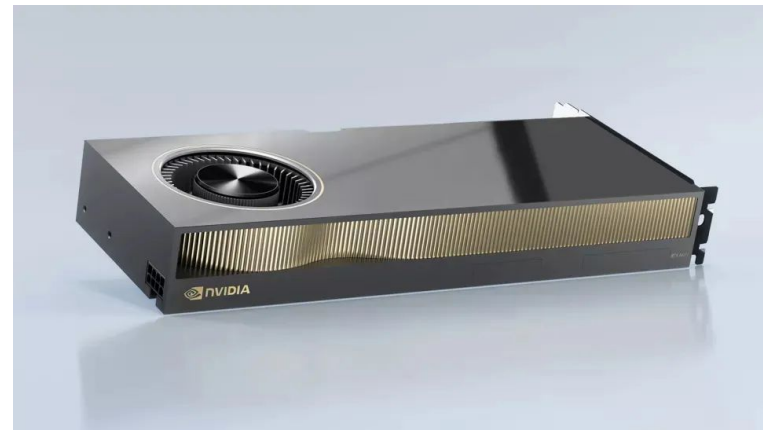
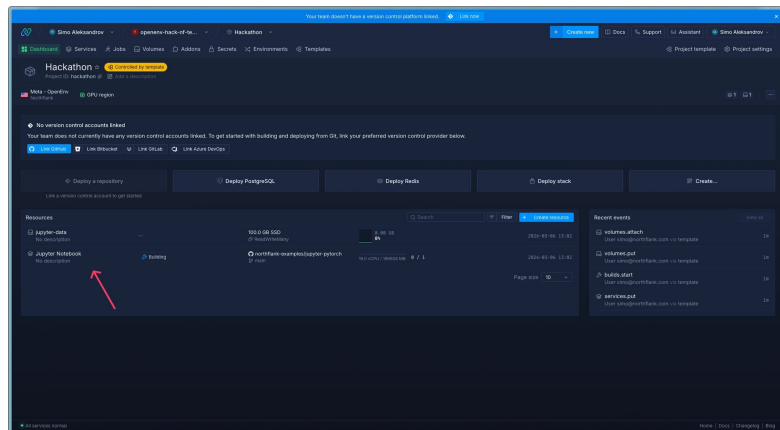
Oumi

veRL

- OpenEnv fundamentals, architecture
- Local dev, Docker, and HF Spaces deployment
- OpenEnv in Practice
- Training (TRL & Unsloth)
- **How-to-Access-Infrastructure**



CoreWeave x Northflank



120 x H100 GPUs
1 GPU per team ~ 120 Teams

References:

GPU Request form: <https://docs.google.com/forms/d/e/1FAIpQLSd2bxx5jAXE8D3FjF7OVekSxwpDVMf1LWE3Z-g4FZoDJ4W6xg/viewform>

Northflank Docs | OpenEnv Hackathon: <https://northflank.notion.site/Northflank-Docs-OpenEnv-Hackathon-2496d14c785180c6a595e6776e3118ba>

Deploy AI projects with Northflank: <https://northflank.notion.site/Deploy-AI-projects-with-Northflank-1a76d14c7851805f8a0ecc780fa33547>

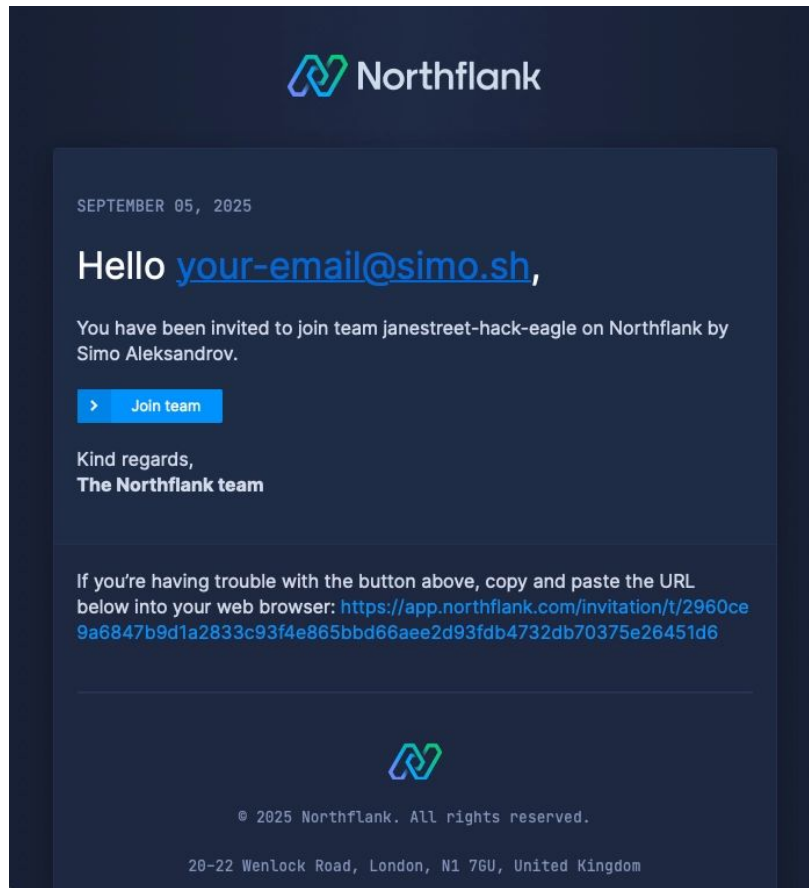
Jupyter Notebook with PyTorch: <https://northflank.notion.site/Jupyter-Notebook-with-PyTorch-2036d14c7851802abb7ccb4a7c5c96be>

Using SSH to access your Services: <https://northflank.notion.site/Using-SSH-to-access-your-Services-3196d14c78518057b05bf61f4e01f55e>

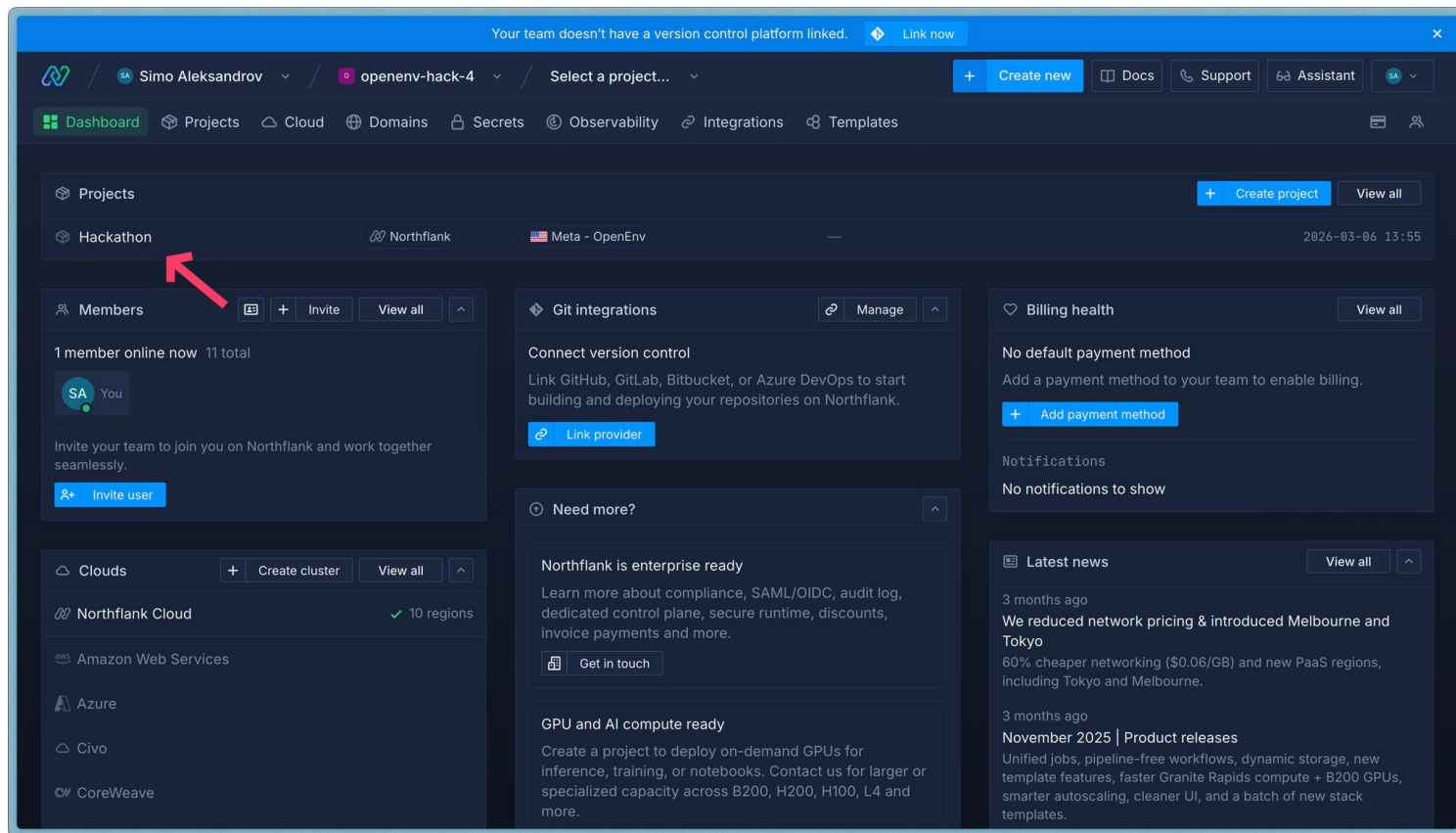
Shell access (VM-like experience): <https://northflank.notion.site/Shell-access-VM-like-experience-1b86d14c785181ac8367c55f68978ae4#1b86d14c785181ca837cf99d37136c41>

Persistent volumes: <https://northflank.notion.site/Persistent-volumes-1a86d14c785180cfaf68dc968f48f3eb>

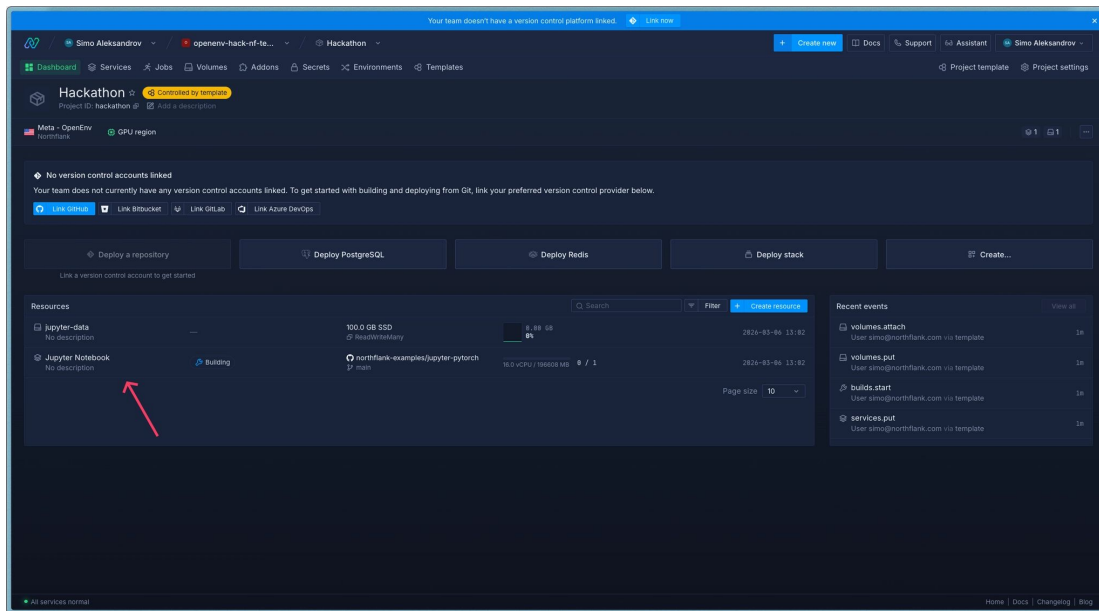
Your team need to do the **GPU Request form**: <https://docs.google.com/forms/d/e/1FAIpQLSd2bxx5jAXE8D3FjF7OVekSxwpDVMf1LWE3Z-g4FZoDJ4W6xg/viewform>
Your team will have a Northflank team and project associated to you. You'll receive an invite for your team to your email address:



Once you've made an account and accepted the invite, you'll be presented with the Northflank home page. Here you can find your project called **Hackathon**. You can click on it and this will navigate you to the project dashboard.



- Your team will have access to a GPU node with a single NVIDIA H100 for your task.
- You can then deploy your code to Northflank, for example
 - **Recommended:** a Jupyter notebook w/ PyTorch preinstalled – [Jupyter Notebook with PyTorch](#)
 - A generic Ubuntu image – [Shell access \(VM-like experience\)](#)
 - Your existing GitHub repository using Docker – [CI/CD Python application](#)
 - Or any Public Docker image – [Common AI Docker Images](#)



- Setup your SSH Key for your project based on what Northflank shared (you need to install Node (npm) and Northflank CLI to ssh from your machine):

<https://northflank.notion.site/Using-SSH-to-access-your-Services-3196d14c78518057b05bf61f4e01f55e#3196d14c785180829665fb64773d0cbc>

```

X northflank (node)
>
~ on ☁ (us-east-1)
> northflank ssh service --proxyOnly
Choose the project to use: metacel
Choose the service to use: openenvtestservice
SSH proxy started on endpoint: 127.0.0.1:38591, stop with Ctrl+C
Use with any SSH compatible client. Example usages:
> SSH session: ssh root@127.0.0.1 -p 38591
> SFTP:      sftp -P 38591 root@127.0.0.1
> RSYNC:     rsync -avz -e "ssh -p 38591" ./data/ root@127.0.0.1:/rsync-test/
> SCP:       scp -P 38591 file.txt root@127.0.0.1:/remote/path/

X root@openenvtestservice-57d4d6c5b4-66g9q: ~ (ssh)
Last login: Fri Mar  6 16:54:51 on ttys039

If intern or devservers are unavailable, use the 'ahno' tool.
'ahno sev0' in SEV0 scenarios, 'ahno devserver' for devserver access
Run 'touch ~/.hushlogin' to suppress this message.

~ on ☁ (us-east-1)
>
~ on ☁ (us-east-1)
>
~ on ☁ (us-east-1)
> ssh -i ~/.ssh/id_northflank -p 38591 root@127.0.0.1
Last login: Sat Mar  7 00:57:02 2026 from 127.0.0.6
root@openenvtestservice-57d4d6c5b4-66g9q:~#
root@openenvtestservice-57d4d6c5b4-66g9q:~#
root@openenvtestservice-57d4d6c5b4-66g9q:~# hostname
openenvtestservice-57d4d6c5b4-66g9q
root@openenvtestservice-57d4d6c5b4-66g9q:~# nvidia-smi

+-----+
| NVIDIA-SMI 580.126.09              Driver Version: 580.126.09      CUDA Version: 13.0     |
+-----+-----+
| GPU   Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           MIG M.         |
+-----+-----+
|  0   NVIDIA H100  80GB HBM3        On      00000000:4C:00.0 Off |   0          0     |
| N/A   24C    P0             68W / 700W |  0MiB / 81559MiB |   0%      Default  |
|                                           Disabled      |
+-----+-----+

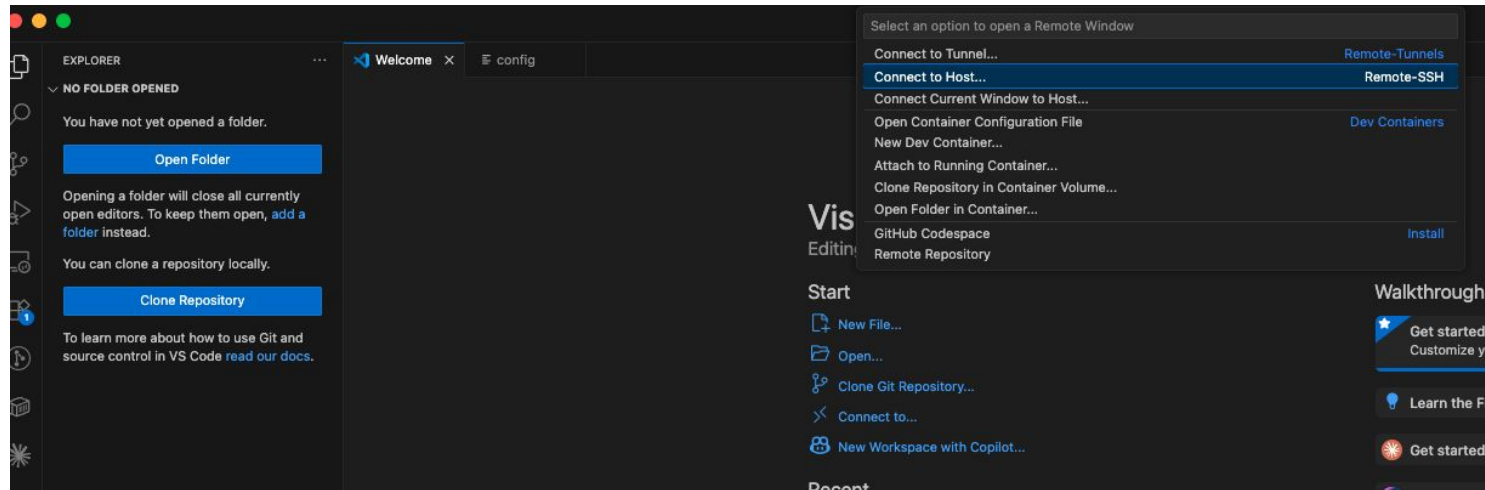
+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
| ID     ID                                   |                 | Usage     |
+-----+-----+
| No running processes found |
+-----+

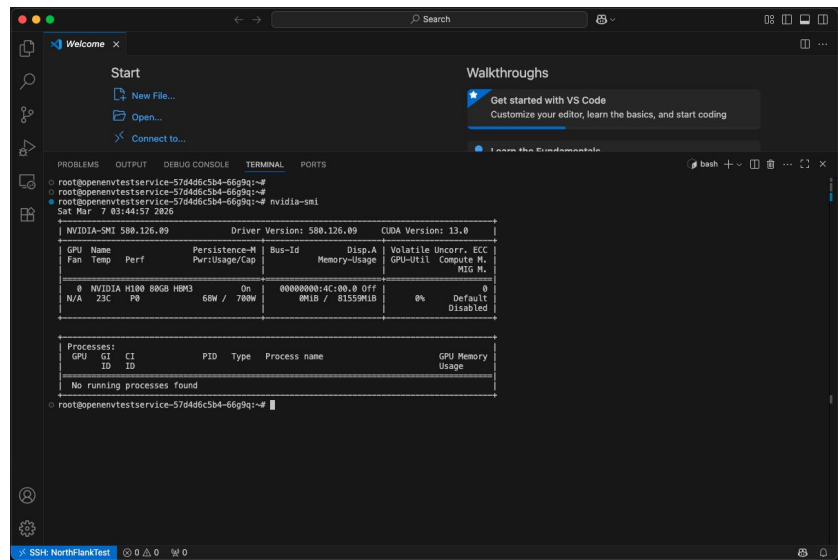
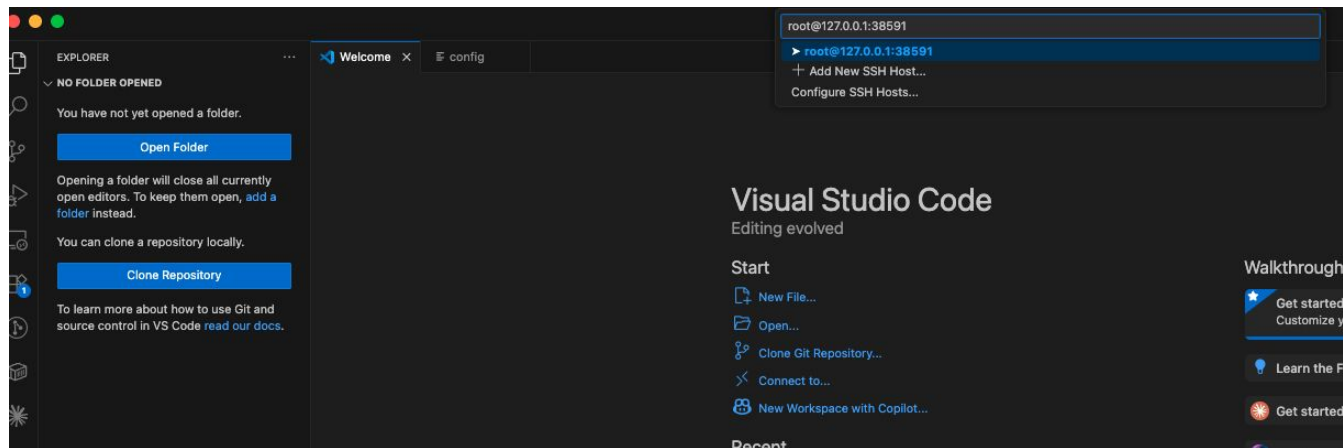
root@openenvtestservice-57d4d6c5b4-66g9q:~#
```


Connect your VSCode to the Container <Service> via SSH

Copy the ip port and root user from the terminal and use it to connect either Cursor or VSCode to the H100 container

```
SSH proxy started on endpoint: 127.0.0.1:38591, stop with Ctrl+C
Use with any SSH compatible client. Example usages:
> SSH session: ssh root@127.0.0.1 -p 38591
> SFTP:         sftp -P 38591 root@127.0.0.1
> RSYNC:        rsync -avz -e "ssh -p 38591" ./data/ root@127.0.0.1:/rsync-test/
> SCP:          scp -P 38591 file.txt root@127.0.0.1:/remote/path/
```





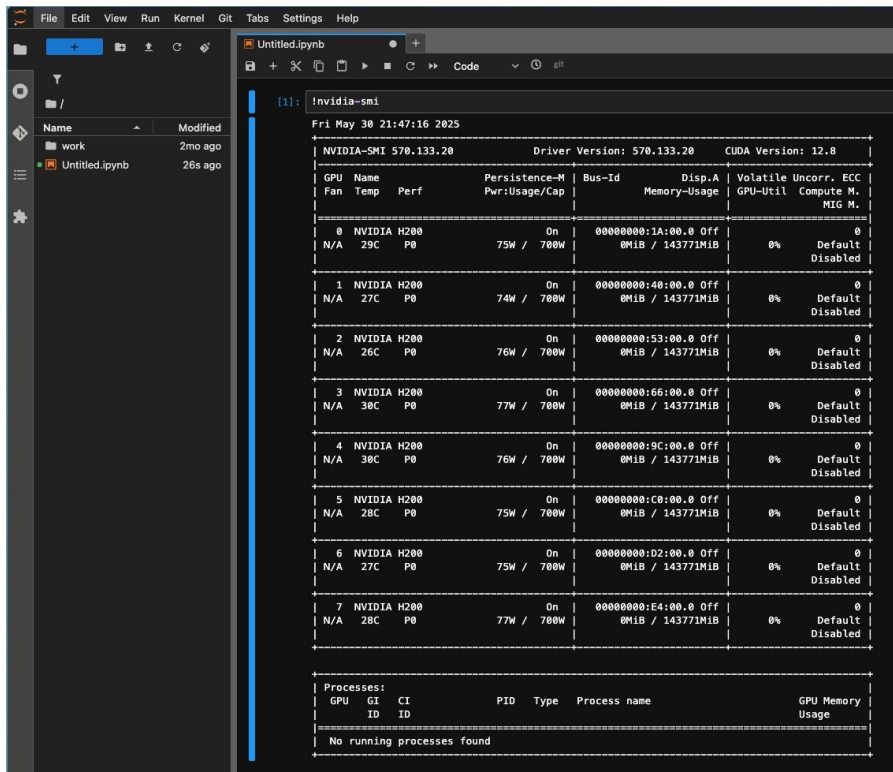
You can set up Jupyter Notebook setup on a service using Northflank one-click deployment:

<https://northflank.notion.site/Jupyter-Notebook-with-PyTorch-2036d14c7851802abb7ccb4a7c5c96be#24c6d14c7851801481f4da71eb2d7d6a>

Once the service is running, you can follow the these steps on How-to-Connect-to-Jupyter-Lab:

<https://northflank.notion.site/Connecting-to-Jupyter-Lab-2036d14c7851809f8eedf5142f4a3eb7>

This is really nice for the Unsloth trainer!
Since the example is provided in a Jupyter Notebook



```
[1]: !nvidia-smi

Fri May 30 21:47:16 2025

+-----+
| NVIDIA-SMI 570.133.20              Driver Version: 570.133.20   CUDA Version: 12.8     |
+-----+-----+
| GPU Name                               Persistence-M   Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp  Perf    Pwr:Usage/Cap       Memory Usage   GPU-Util    Compute M. |
|=====+=====+
| 0  NVIDIA H200              P0              On          00000000:1A:00.0 Off |
| N/A   29C    P0               75W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 1  NVIDIA H200              P0              On          00000000:40:00.0 Off |
| N/A   27C    P0               74W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 2  NVIDIA H200              P0              On          00000000:53:00.0 Off |
| N/A   26C    P0               76W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 3  NVIDIA H200              P0              On          00000000:66:00.0 Off |
| N/A   30C    P0               77W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 4  NVIDIA H200              P0              On          00000000:9C:00.0 Off |
| N/A   30C    P0               76W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 5  NVIDIA H200              P0              On          00000000:C0:00.0 Off |
| N/A   28C    P0               75W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 6  NVIDIA H200              P0              On          00000000:D2:00.0 Off |
| N/A   27C    P0               75W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+
| 7  NVIDIA H200              P0              On          00000000:E4:00.0 Off |
| N/A   28C    P0               77W / 700W           0MiB / 143771MiB      0%      Default |
|                                     Disabled |
+-----+-----+

+-----+
| Processes:                               |
| GPU   GI   CI          PID    Type   Process name                        GPU Memory |
| ID   ID                                   |           |
+-----+-----+
| No running processes found              |           |
+-----+
```

Still have Questions?

**Please mention it in discord and we do
our best to answer**